6

Adding Authentication with JWT

After developing and deploying our first full-stack application, we now have a way for anyone to create posts on our blog. However, since the author is an input field, anyone could enter any author, impersonating others! That's not good. In this chapter, we are going to add authentication with **JSON Web Token** (**JWT**) and functionalities to sign up and log into our application by adding additional routes using React Router.

In this chapter, we are going to cover the following main topics:

- What is JWT?
- Implementing login, signup, and authenticated routes in the backend using JWT
- Integrating login and signup in the frontend using React Router and JWT
- Advanced token handling

Technical requirements

Before we start, please install all the requirements from ***Chapter 1***, *Preparing for Full-Stack Development*, and ***Chapter 2***, *Getting to Know Node.js and MongoDB*.

The versions listed in those chapters are the ones used in this book. While installing a newer version should not be an issue, please note that certain steps might work differently. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in ***Chapter 1*** and ***Chapter 2***.

You can find the code for this chapter on GitHub: **https://github.com/PacktPublishing/Modern-Full-Stack-React-**

Projects/tree/main/ch6.

The CiA video for this chapter can be found at: https://youtu.be/LloHmkgRLWk.

What is JWT?

JWT, pronounced "jot", is an open industry standard (RFC 7519) for safely passing claims between multiple parties. Claims can be information about a certain party or object, such as the email address, user ID, and roles of a user. In our case, we will pass JWTs between our backend and frontend.

JWT is used by many products and services and is supported by third-party authentication providers, such as Auth0, Okta, and Firebase Auth. It is easy to parse JWTs as we only need to base64 decode them and parse the JSON string. After verifying the signature, we can be sure that the JWT is authentic and trust the claims within it.

JWTs consist of the following components:

- **Header**: Containing the algorithm and token type
- **Payload**: Containing the data/claims of the token
- **Signature**: For verifying that the token was created by a legit source

These three components form a JWT as they're joined into a single string, separated by a period (`.`), as follows:

```
header.payload.signature
```

Let's look at each component separately.

JWT header

The JWT header typically consists of a token type (in our case, JWT), specified by the **`typ`** property, and the algorithm used to create the signature (in our case, we will use HMAC SHA256, a SHA256 hash-based message authentication code), specified by the **`alg`** property. The header is defined as a JSON object, like so:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

This JSON object is then base64 encoded and forms the first part of the JWT.

JWT payload

The main part of the JWT is the payload, which contains all claims. Claims are information about an entity (such as the user) and additional data. The JWT standard distinguishes between three types of claims:

- **Registered claims**: These are predefined claims and it's recommended that they're set. They include information about the following:
  - The issuer (`iss`), which is the entity that created the token.
  - The expiration time (`exp`), which tells us when the token expires.
  - The subject (`sub`), which tells us about the entity identified by the token (such as the user who generated the token during a login).
  - The audience (`aud`), which tells us about the intended recipients of the token.
  - The issued at time (`iat`), which tells us when the token was created.
  - The not before time (`nbf`), which specifies a time before which the token is not valid yet.
  - The JWT ID (`jti`), which provides a unique identifier for the JWT. It's used to prevent JWTs from being replayed.

NOTE

The JSON object properties defined in the JWT standard are all three-letter names to keep the JWT as compact as possible.

- **Public claims**: These are additional claims that are commonly used and shared across many services. A list of those can be found on the **Internet Assigned Numbers Authority** (**IANA**) website: **https://www.iana.org/assignments/jwt/jwt.xhtml**. If we want to store additional information, we should always consult this list first to see if we can use a standardized claim name.

- **Private claims**: These are custom-defined claims, which are neither registered nor public. If we need a special claim that isn't defined yet, we can make a private claim that only our services will understand.

All claims are optional, but it makes sense to at least include one claim to identify the subject, such as the **`sub`** registered claim.

Putting together what we've learned, we can create the following example payload:

```
{
  "sub": "1234567890",
  "name": "Daniel Bugl",
  "admin": true
}
```

In our example, the **`sub`** claim is a registered claim, the **`name`** claim is a public claim, and the **`admin`** claim is a private claim.

The payload is also base64 encoded and forms the second part of the JWT. As such, this information is publicly readable by anyone who has access to the token. Do not put secret information into the payload or header of a JWT! However, the information cannot be *changed* without invalidating the existing signature, making all claims tamper-proof. Only a backend service with access to the private key can generate a new signature to create a valid JWT.

JWT signature

The final part of a JWT is its signature. The signature is what proves that all the information that we've defined up until now has not been tampered with. The signature is created by taking the base64-encoded header and payload, joining those strings with a period symbol, and using the specified algorithm to sign it with a secret key:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret
)
```

Now that we've learned about the different components of a JWT, let's put this all together to create a valid JWT.

Creating a JWT

Follow these steps to create a JWT:

1. Go to the **https://jwt.io/** website and scroll down to the **Debugger** section.
2. Enter our previously defined header and payload.
3. Enter **`full-stack`** as the secret.
4. The encoded JWT should update on the fly as you're changing the values.

As you can see, we have successfully created our first JWT:

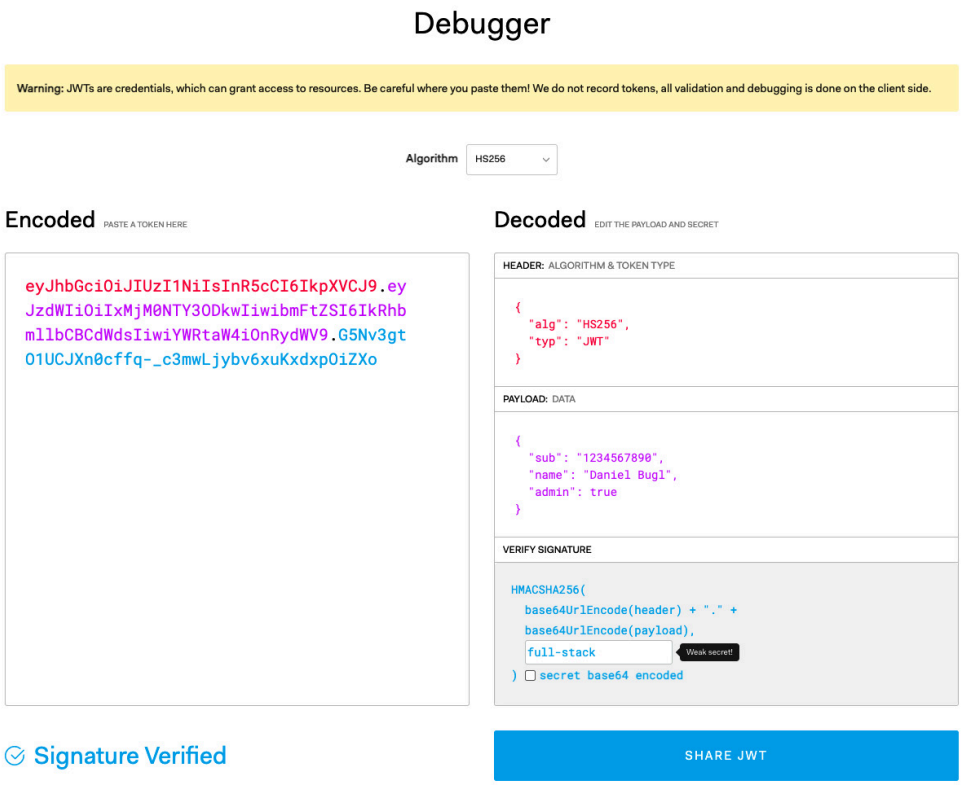

Figure 6.1 – Our first JWT, created with the jwt.io debugger

The generated JWT consists of three components, each of them base64-encoded and separated by periods. In the debugger, they are highlighted in three different colors. Try changing the base64 string in the "Encoded" section by removing some characters; you will see that the JWT is now invalid due to an "Invalid Signature" issue. Now that we've created our first JWT, let's learn how to use it.

Using JWT

In the login process, we are going to generate a JWT for the logged-in user in the backend. This JWT will be returned to the user's browser. When the user wants to access a protected route, we can send the JWT to the backend server by using the **`Authorization`** header with the **`Bearer`** schema, as follows:

```
Authorization: Bearer <token>
```

The backend can then check for this header, verify the signature of the token, and grant the user access to certain routes. By sending the token in a header instead of a cookie, we don't have to deal with CORS issues that we would have when dealing with cookies.

NOTE

Be careful not to send too much data in the header since some servers do not accept more than 8 KB in headers. This means that, for example, complex role information should not be stored in the JWT claims as it might take up too much space. Instead, this kind of information could be stored in the database associated with a user ID from the JWT.

An interesting advantage of using a JWT is that the authentication server and the actual backend for our app do not have to be the same. We could have a separate authentication service, get a JWT, and in the backend verify the signature of the JWTs to guarantee that they were generated by the authentication service. This allows us to use external services for authentication, such as Auth0, Okta, or Firebase Auth.

The following diagram shows the authorization flow for a JWT:

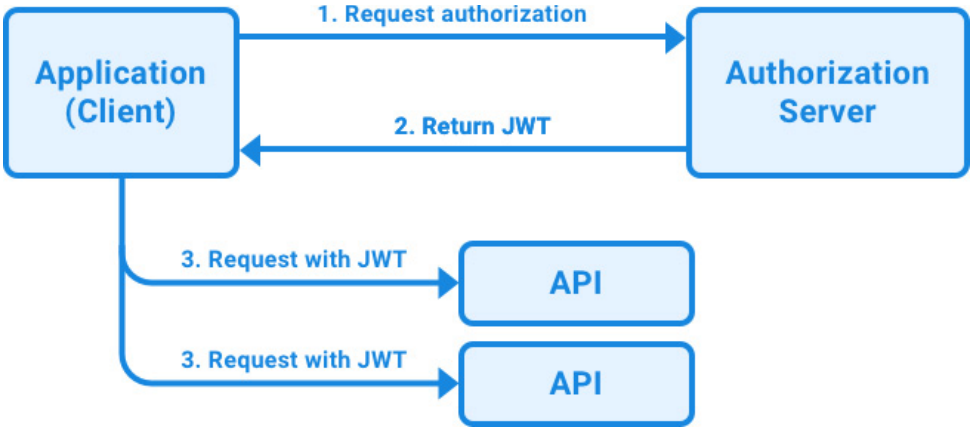

Figure 6.2 – Authorization flow for a JWT

As we can see, the application requests authorization to the authorization server, which can also be either a third-party provider, a separate service, or part of the backend service. Then, when authorization is granted (if the login details are correct), the authorization server returns a JWT. This JWT can then be used to access protected routes on APIs. Before granting access, the JWT signature is validated to ensure that it has not been tampered with.

Storing JWT

We should take great care about where we store the JWT. Local storage is *not* a good way to store authentication information such as a JWT. Cross-site scripting can be used to steal all data in local storage. For short-lived tokens, we can store them in a JavaScript runtime variable (such as a React context). For longer-term storage, we could use an **`httpOnly`** cookie, which has additional security guarantees.

Now that we've learned how JWT works, let's put theory into practice and implement login, signup, and authenticated routes in the backend using JWT.

Implementing login, signup, and authenticated routes in the backend using JWTs

Now that we've learned about JWTs, we'll implement them in our backend. First, we need to create a user model in our database, after which we

can create routes to sign up and log into our app. Finally, we will implement authenticated routes that are only accessible with a JWT.

Creating the user model

We'll start the backend implementation by creating a user model, as follows:

1. Copy the **ch5** folder to a new **ch6** folder, as follows:

```
$ cp -R ch5 ch6
```

2. Open the **ch6** folder in VS Code.
3. Create a new **backend/src/db/models/user.js** file and define a new **userSchema** inside it:

```
import mongoose, { Schema } from 'mongoose'
const userSchema = new Schema({
```

4. A user should have a required unique **username** and a required **password**:

```
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
})
```

5. Create and export the model:

```
export const User = mongoose.model('user', userSchema)
```

6. At this point, let's also adjust the post model so that we can store a reference to a user ID instead of the username as the author. Edit **backend/src/db/models/post.js**, as follows:

```
    author: { type: Schema.Types.ObjectId, ref: 'user', required: true },
```

We changed the type to **ObjectId**, with a reference to the **user** model, and made **author** required (as you will need to be logged in to create a post after we add an authenticated route later in this chapter).
Making **author** required means that the unit tests will need to be adjusted, but doing so is left as an exercise for you.

Now that we've successfully created the user model, let's move on to creating the signup service so that we have a way to create new users.

Creating the signup service

When a user signs up, we need to hash the password provided by the user before storing it in the database. We should never store passwords in plaintext as that would mean that if our database gets leaked, an attacker will have access to the passwords of all users. Hashing is a one-way function that turns a string into a different string in a deterministic way. This means that, for example, if we do **`hash("password1")`**, we get a specific string every time we do it. However, if we do **`hash("password2")`**, we get a completely different string. By choosing a good hash function, we can ensure that reversing a hash is so computationally expensive that it is impossible to do in a reasonable time. When the user signs up, we can store the hash of their password. When a user then enters their password to log in, we can hash their entered password again and compare it to the hash in the database.

Let's start implementing the signup service with hashed passwords:

1. Install the **`bcrypt`** npm package. We are going to use this to hash the password before storing it:

```
$ cd backend
$ npm install bcrypt@5.1.1
```

2. Create a new **`backend/src/services/users.js`** file and import **`bcrypt`** and the **`User`** model:

```
import bcrypt from 'bcrypt'
import { User } from '../db/models/user.js'
```

3. Define a **`createUser`** function that takes **`username`** and **`password`** values:

```
export async function createUser({ username, password }) {
```

4. Inside this function, we use the **`bcrypt.hash`** function to create a hash from the plaintext password using 10 salt rounds (repeating the hashing 10 times to make it even harder to reverse it):

```
  const hashedPassword = await bcrypt.hash(password, 10)
```

5. Now, we can create a new user and store it in our database:

```
  const user = new User({ username, password: hashedPassword })
  return await user.save()
}
```

For brevity, we won't cover creating tests for the user services. Refer to ***Chapter 3***, *Implementing a Backend Service Using Express, Mongoose ODM, and Jest*, for information on how to create tests for your service functions. You can write similar tests to what we did for the posts service functions.

After creating the signup service, we can create the signup route.

Creating the signup route

Now, let's expose the signup service function by adding an API route for it:

1. Create a new **`backend/src/routes/users.js`** file and import the **`createUser`** service:

```
import { createUser } from '../services/users.js'
```

2. Define a new **`userRoutes`** function and expose a **`POST /api/v1/user/signup`** route. This route creates a new user from the request body and return the username:

```
export function userRoutes(app) {
  app.post('/api/v1/user/signup', async (req, res) => {
    try {
      const user = await createUser(req.body)
      return res.status(201).json({ username: user.username })
    } catch (err) {
      return res.status(400).json({
        error: 'failed to create the user, does the username already exist?'
      })
    }
```

```
  })
}
```

In this case, we define a singular **`user`** route instead of calling it **`users`** as we are only dealing with one user at a time. To keep things simple, the error handling is very rudimentary. It would be a good idea to distinguish between the different errors that can happen and show a different error message, depending on the error.

3. Edit **`backend/src/app.js`** and import the **`userRoutes`** function:

```
import { postRoutes } from './routes/posts.js'
import { userRoutes } from './routes/users.js'
```

4. In the same file, call the **`userRoutes`** function after the **`postRoutes`** function to mount them:

```
postRoutes(app)
userRoutes(app)
```

5. Make sure the **`dbserver`** container is running in Docker.
6. Start the backend by running the following command in a Terminal inside the **`backend/`** folder:

```
$ cd backend
$ npm run dev
```

7. Now, make a request to the new **`POST /api/v1/user/signup`** route. You will see that creating a user works if **`username`** and **`password`** values are provided properly. Enter the following code in your browser console while the backend is running, on a blank tab or at **`http://localhost:3001/`**:

```
const res = await fetch('http://localhost:3001/api/v1/user/signup', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: 'dan', password: 'hunter2' })
})
console.log(await res.json())
```

8. If we try creating another user with the same username (by executing the same fetch again), it will fail because the **`username`** field is defined to be unique in Mongoose.

Now that we have successfully created our first user, let's continue by creating the login service to allow our user to log in.

Creating the login service

So far, we have only created a user in our database. As we aren't authorizing the user yet, we haven't dealt with JWTs yet. Let's start doing that now:

1. Open a new Terminal and install the **`jsonwebtoken`** library, which contains functions to deal with the creation and verification of JWTs:

```
$ cd backend
$ npm install jsonwebtoken@9.0.2
```

2. Edit the **`backend/src/services/users.js`** file and import **`jwt`** from the **`jsonwebtoken`** library:

```
import jwt from 'jsonwebtoken'
```

3. Define a new **`loginUser`** function, which takes a username and password:

```
export async function loginUser({ username, password }) {
```

4. Now, fetch a user with the given **`username`** from our database:

```
  const user = await User.findOne({ username })
  if (!user) {
    throw new Error('invalid username!')
  }
```

5. Then, use **`bcrypt.compare`** to compare the entered password to the hashed password from the database:

```
  const isPasswordCorrect = await bcrypt.compare(password, user.password)
  if (!isPasswordCorrect) {
    throw new Error('invalid password!')
  }
```

6. If the user correctly enters a username and password, we use **`jwt.sign()`** to create a new JWT and sign it with a secret. For the secret, we use an environment variable:

```
  const token = jwt.sign({ sub: user._id }, process.env.JWT_SECRET, {
    expiresIn: '24h',
  })
```

In the last argument, we also specify that our token should be valid for 24 hours.

NOTE

We are using the user ID, not the username, to identify the user. This is done to future-proof the system as the user ID is a value that will never change. In the future, we might want to add a way to change the username. It would be hard to deal with such a change if we always use the username to identify the user.

7. Lastly, we return the token:

```
  return token
}
```

8. Now, we define the **`JWT_SECRET`** environment variable by editing the **`.env`** file:

```
JWT_SECRET=replace-with-random-secret
```

Make sure you generate a safe JWT secret for the production environment, which you never expose or use in development environments or for debugging! If you want to deploy your app to Google Cloud Run again, you would also need to add this secret as an environment variable there.

9. We'll also add one to **`.env.template`** as an example:

```
JWT_SECRET=replace-with-random-secret
```

After successfully creating a login service to create and sign JWTs, we can create the login route.

Creating the login route

We still need to expose the login service as an API route for users to be able to log in. Let's do that now:

1. Edit the **`backend/src/routes/users.js`** file and import the **`loginUser`** function:

```
import { createUser, loginUser } from '../services/users.js'
```

2. Add a new **`POST /api/v1/user/login`** route inside the **`userRoutes`** function, where we call the **`loginUser`** function and return the token:

```
  app.post('/api/v1/user/login', async (req, res) => {
    try {
      const token = await loginUser(req.body)
      return res.status(200).send({ token })
    } catch (err) {
      return res.status(400).send({
        error: 'login failed, did you enter the correct username/password?'
      })
    }
  })
```

3. If the backend is not running anymore, start it again. Then, make a request to **`/api/v1/user/login`** to test it out by entering the following code in your browser console:

```
const res = await fetch('http://localhost:3001/api/v1/user/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: 'dan', password: 'hunter2' })
})
console.log(await res.json())
```

4. We have successfully created a valid JWT! To verify that the JWT is valid, we can paste it into the debugger at **https://jwt.io/**. Make sure that you also change the secret in the **Verify Signature** section on the page, as shown in the following screenshot:

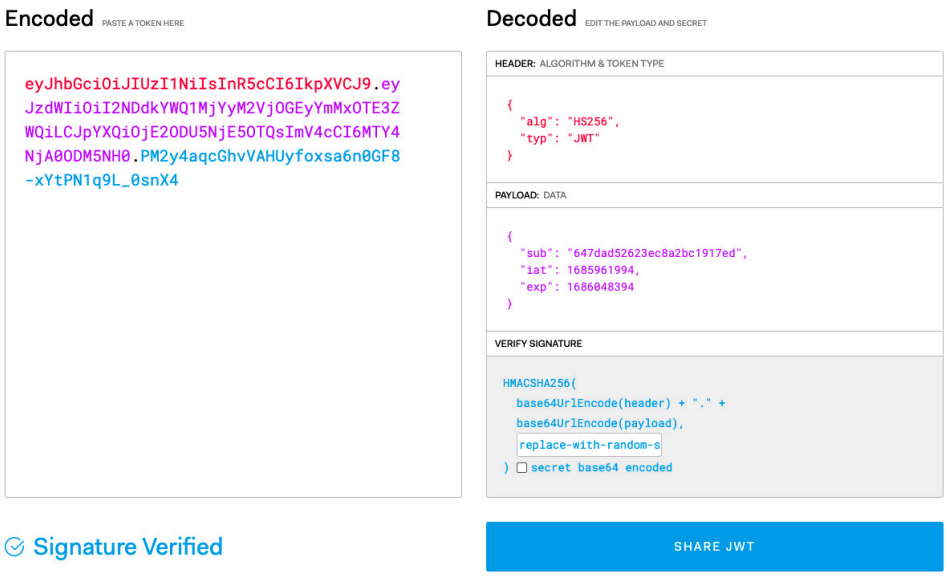

Figure 6.3 – Verifying the JWT created from the login service

NOTE

When copying the token from the JSON response in your browser, make sure that you are copying the full string value, and not the truncated one (with ... in the middle of the string). Otherwise, the JWT might not decode properly in the debugger.

After successfully logging our user in and creating a token for them, we can now protect certain routes and make sure that only logged-in users can access them.

Defining authenticated routes

Now that we have successfully created a valid JWT, we can start protecting routes. To do so, we are going to use the **`express-jwt`** library, as follows:

1. Install the **`express-jwt`** npm package:

```
$ cd backend
$ npm install express-jwt@8.4.1
```

2. Create a new **`backend/src/middleware`** folder. Inside it, create a new **`backend/src/middleware/jwt.js`** file and import **`expressjwt`** there:

```
import { expressjwt } from 'express-jwt'
```

3. Create and export a **`requireAuth`** middleware by using the **`expressjwt`** function and your secret and algorithm settings:

```
export const requireAuth = expressjwt({
  secret: () => process.env.JWT_SECRET,
  algorithms: ['HS256'],
})
```

   We need to use a function for the secret because **`dotenv`** isn't initialized at import time yet, so the environment variable will only be available later. Specifying the algorithms is required to prevent potential downgrade attacks.

4. Edit **`backend/src/routes/posts.js`** and import the **`requireAuth`** middleware:

```
import { requireAuth } from '../middleware/jwt.js'
```

5. Add the middleware to the create route. Middleware in Express can be added to specific routes by passing it as a second argument to the function, as follows:

```
  app.post('/api/v1/posts', requireAuth, async (req, res) => {
```

6. Repeat the same for the edit route:

```
  app.patch('/api/v1/posts/:id', requireAuth, async (req, res) => {
```

7. Now, do this for the delete route:

```
  app.delete('/api/v1/posts/:id', requireAuth, async (req, res) => {
```

8. Try accessing the routes without being logged in. You will see that they fail with a **401 Unauthorized** status. Execute the following code into your browser console:

```
const res = await fetch('http://localhost:3001/api/v1/posts', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({ title: 'Test Post' })
})
console.log(await res.json())
```

You can see the results of executing the code in the following screenshot:

Figure 6.4 – Attempting to access a protected route without a JWT and then with a JWT

NOTE

Instead of using the **`express-jwt`** *library, we could also manually extract the token from the* **`Authorization`** *header and use the* **`jwt.verify`** *function from the* **`jsonwebtoken`** *library to verify it.*

The routes are protected now, but we aren't considering which user accessed them. Let's do that now by accessing the currently logged-in user from the token.

Accessing the currently logged-in user

After adding authenticated routes, we successfully protected some routes so that they can only be accessed by logged-in users. However, it's still possible to edit posts of other users or create posts under a different username. Let's change that:

1. Edit the **`backend/src/services/posts.js`** file and add a **`userId`** argument to the **`createPost`** function, *removing* **`author`** from the object:

```
export async function createPost(userId, { title, author, contents, tags }) {
```

2. Instead of setting the author through the request body, we will set the author to the ID of the logged-in user:

```
  const post = new Post({ title, author: userId, contents, tags })
```

3. We adjust the **`updatePost`** and **`deletePost`** functions similarly (adding the **`userId`** argument, removing the **`author`** argument, and removing the author variable from the **`$set`** object), ensuring that the currently logged-in user is the author of the post:

```
export async function updatePost(userId, postId, { title, author, contents, tags
  return await Post.findOneAndUpdate(
    { _id: postId, author: userId },
    { $set: { title, author, contents, tags } },
    { new: true },
  )
}
export async function deletePost(userId, postId) {
  return await Post.deleteOne({ _id: postId, author: userId })
}
```

   In our case, we simply fetch a post with the given ID and an author as the current user. We could still extend this code to first fetch the post with the given ID, check if it exists (if not, return a **404 Not Found** error), and if it does exist, verify that the author is the currently logged-in user (if not, return a **403 Forbidden** error).

NOTE

This is a breaking API change and requires changing the tests. For brevity, we will not go through adjusting the tests step by step here, so this is left as an exercise for you.

4. Edit the **`backend/src/routes/posts.js`** file and use the **`req.auth.sub`** variable to pass the user ID to the **`createPost`** function:

```
    const post = await createPost(req.auth.sub, req.body)
```

5. Do the same for the **`updatePost`** function:

```
```

```
    const post = await updatePost(req.auth.sub, req.params.id, req.body)
```

6. Also, do this for the **`deletePost`** function:

```
    const { deletedCount } = await deletePost(req.auth.sub, req.params.id)
```

7. Try creating a new post; you will see that it is created by the user identified in the JWT. You can do this by executing the following code in the browser console (don't forget to replace **`<TOKEN>`** with your previously generated JWT):

```
const res = await fetch('http://localhost:3001/api/v1/posts', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'Authorization': 'Bearer <TOKEN>'
    },
    body: JSON.stringify({ title: 'Test Post' })
})
console.log(await res.json())
```

   Editing and deleting your posts is also possible, but not for posts from other users anymore!

INFO

The **`express-jwt`** *middleware stores all decoded claims from the JWT in a* **`req.auth`** *object. So, we can access any claims made when creating our JWT here. Of course, the middleware validates the JWT signature against the defined secret first, to ensure that it received an authentic JWT.*

Now that we've set up the login, signup, and authenticated routes, let's continue by integrating login and signup in the frontend.

Integrating login and signup in the frontend using React Router and JWT

Now that we have successfully implemented authorization in the backend, let's start extending the frontend with signup and login pages and connecting them to the backend. First, we are going to learn how to implement multiple pages in a React app using React Router. Then, we are

going to implement the signup UI and connect it to the backend. Afterward, we are going to implement a login UI, store the token in the frontend, and set up automatic redirects when we are successfully logged in. Finally, we are going to update the code for creating posts to pass the token in the Authorization header and properly access our authenticated route.

Let's get started with the frontend integration by setting up React Router.

Using React Router to implement multiple routes

React Router is a library that allows us to manage routing in our app by defining multiple pages on different routes, just like what we have done in Express for API routes, but for the frontend! Let's set up React Router:

1. Install the **`react-router-dom`** library in the frontend project (the root of the **`ch6`** folder, not inside the **`backend`** folder):

```
$ npm install react-router-dom@6.21.0
```

2. Edit **`src/App.jsx`** and import the **`createBrowserRouter`** function and **`RouterProvider`** component:

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom'
```

3. Create a new **`router`** and define the routes. First, we'll define an index route for rendering our **`Blog`** component:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Blog />,
  },
])
```

4. Then, in the **`App`** component, replace the **`<Blog>`** component with **`<RouterProvider>`**, as follows:

```
export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <RouterProvider router={router} />
```

```
      </QueryClientProvider>
    )
  }
```

5. Start the frontend by running the following command in the root of the **ch6** folder:

```
$ npm run dev
```

6. The blog should render the same way as before, but now, we can start defining new routes! You can verify that React Router is working by going to a page that we did not define – for example, **http://localhost:5173/test**. React Router will display the default 404 page, as shown in the following screenshot:

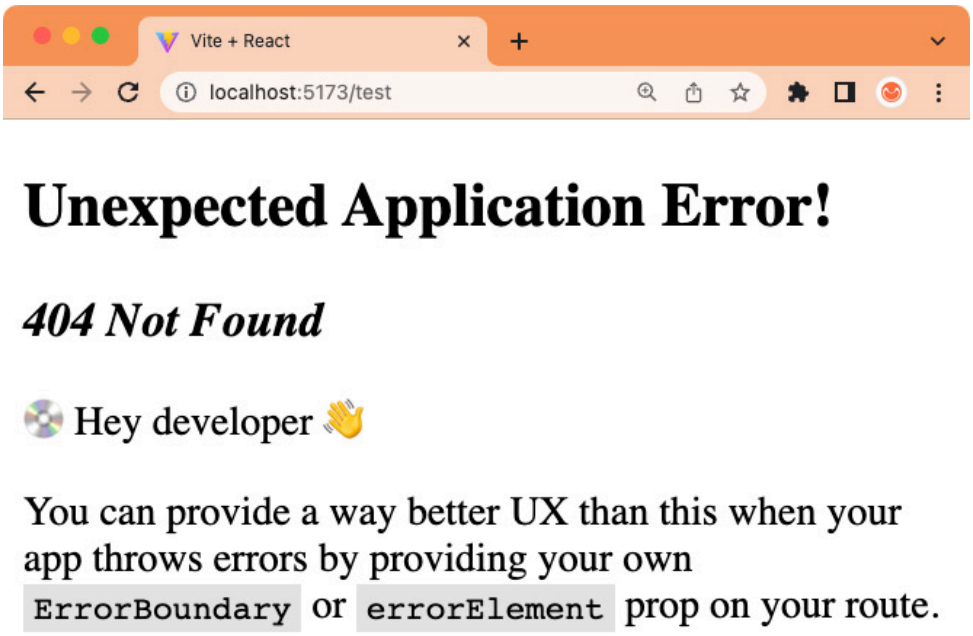

Figure 6.5 – The default 404 page provided by React Router

Now that we have successfully set up React Router, we can move on to creating the signup page.

Creating the signup page

We will start by updating our folder structure so that it supports multiple pages. Then, we will implement a **Signup** component and define a **/signup** route to link to it. Follow these steps:

1. Create a new **src/pages/** folder.
2. Move the **src/Blog.jsx** file into the **src/pages/** folder. When VS Code asks you to update all imports, select **Yes**. Alternatively, update the im-

port in **src/App.jsx**, as follows:

```
import { Blog } from './pages/Blog.jsx'
```

3. Create a new **src/api/users.js** file and define an API function for the **signup** route, as follows:

```
export const signup = async ({ username, password }) => {
  const res = await fetch(`${import.meta.env.VITE_BACKEND_URL}/user/signup`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password }),
  })
  if (!res.ok) throw new Error('failed to sign up')
  return await res.json()
}
```

We are checking for **res.ok** here, which will be **false** when the response status code is an error code, such as **400**.

4. Create a new **src/pages/Signup.jsx** file, import the **useState**, **useMutation**, and **useNavigate** hooks from **react-router-dom**, as well as the **signup** function, and define a **Signup** component there:

```
import { useState } from 'react'
import { useMutation } from '@tanstack/react-query'
import { useNavigate } from 'react-router-dom'
import { signup } from '../api/users.js'
export function Signup() {
```

5. In this component, we first create state hooks for the **username** and **password** fields:

```
  const [username, setUsername] = useState('')
  const [password, setPassword] = useState('')
```

6. Then, we use the **useNavigate** hook to get a function to navigate to a different route:

```
  const navigate = useNavigate()
```

7. We also define a **useMutation** hook to send the **signup** request. On success, we navigate to the **/login** route, which we will define soon:

```
  const signupMutation = useMutation({
```

```
    mutationFn: () => signup({ username, password }),
    onSuccess: () => navigate('/login'),
    onError: () => alert('failed to sign up!'),
  })
```

In case of an error, we could also use the **`signupMutation.isError`** state and the response from the backend to show a more nicely formatted error message.

8. Then, we define a function to handle the submission of the form, as we did for the **`CreatePost`** component:

```
const handleSubmit = (e) => {
  e.preventDefault()
  signupMutation.mutate()
}
```

9. Now, we create a simple form to enter a username, password, and a button to submit the request, similar to the **`CreatePost`** component:

```
return (
  <form onSubmit={handleSubmit}>
    <div>
      <label htmlFor='create-username'>Username: </label>
      <input
        type='text'
        name='create-username'
        id='create-username'
        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />
    </div>
    <br />
    <div>
      <label htmlFor='create-password'>Password: </label>
      <input
        type='password'
        name='create-password'
        id='create-password'
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
    </div>
    <br />
    <input
      type='submit'
      value={signupMutation.isPending ? 'Signing up...' : 'Sign Up'}
```

```
          disabled={!username || !password || signupMutation.isPending}
        />
      </form>
    )
  }
```

10. Edit **`src/App.jsx`** and import the **`Signup`** page component:

```
import { Signup } from './pages/Signup.jsx'
```

11. Add a new **`/signup`** route that points to the **`Signup`** page component:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Blog />,
  },
  {
    path: '/signup',
    element: <Signup />,
  },
])
```

After defining the signup page, we still need a way to link to it. Let's add the link now.

Linking to other routes using the Link component

Now that we have multiple pages in our blog app, we need to link between them. To do this, we can use the **`Link`** component provided by React Router. We could also use a normal link by using **`<a href="">`**, but that would cause a full page refresh. The **`Link`** component uses client-side routing and thus avoids doing a full refresh of the page. Instead, it immediately renders the new component on the client side.

Follow these steps to create a link from the index page to the signup page:

1. Create a new **`src/components/Header.jsx`** file and import the **`Link`** component from **`react-router-dom`**:

```
import { Link } from 'react-router-dom'
```

2. Define a component and return the **`Link`** component to define a link to the signup route, as follows:

```
export function Header() {
  return (
    <div>
      <Link to='/signup'>Sign Up</Link>
    </div>
  )
}
```

3. Edit **`src/pages/Blog.jsx`** and import the **`Header`** component:

```
import { Header } from '../components/Header.jsx'
```

4. Then, render the **`Header`** component in the **`Blog`** component:

```
  return (
    <div style={{ padding: 8 }}>
      <Header />
      <br />
      <hr />
      <br />
      <CreatePost />
```

5. Edit **`src/pages/Signup.jsx`** and import the **`Link`** component:

```
import { useNavigate, Link } from 'react-router-dom'
```

6. Add the **`Link`** component to link back to the index page:

```
  return (
    <form onSubmit={handleSubmit}>
      <Link to='/'>Back to main page</Link>
      <hr />
      <br />
```

Now that we've successfully linked our signup page, let's continue by creating the login page.

Creating the login page and storing the JWT

Now that we have successfully defined the signup page, we can create the login page. However, first, we need to come up with a way to store the

JWT. We shouldn't store it in local storage as a potential attacker can steal the token from there (through, for example, script injection). In a **single-page application** (**SPA**), where we have no page reloads, a safe and simple way to store the token is to store it in the runtime using a React context. Let's do that now:

1. Create a new **`src/contexts/`** folder. Inside it, create a **`src/contexts/AuthContext.jsx`** file and import the **`createContext`**, **`useState`**, and **`useContext`** functions from **`react`**:

```
import { createContext, useState, useContext } from 'react'
import PropTypes from 'prop-types'
```

2. Then, define the following context:

```
export const AuthContext = createContext({
  token: null,
  setToken: () => {},
})
```

3. Next, define an **`AuthContextProvider`** component that provides the context with a state hook:

```
export const AuthContextProvider = ({ children }) => {
  const [token, setToken] = useState(null)
  return (
    <AuthContext.Provider value={{ token, setToken }}>
      {children}
    </AuthContext.Provider>
  )
}
AuthContextProvider.propTypes = {
  children: PropTypes.element.isRequired,
}
```

4. Also, define a hook to use the context with a **`useState`**-like API:

```
export function useAuth() {
  const { token, setToken } = useContext(AuthContext)
  return [token, setToken]
}
```

5. Edit **`src/App.jsx`** and import **`AuthContextProvider`**:

```
import { AuthContextProvider } from './contexts/AuthContext.jsx'
```

6. Wrap **`RouterProvider`** with **`AuthContextProvider`** to make it available to all pages:

```
      <AuthContextProvider>
        <RouterProvider router={router} />
      </AuthContextProvider>
```

7. Edit **`src/api/users.js`** and define a new login function:

```
export const login = async ({ username, password }) => {
  const res = await fetch(`${import.meta.env.VITE_BACKEND_URL}/user/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password }),
  })
  if (!res.ok) throw new Error('failed to login')
  return await res.json()
}
```

8. Copy over the **`src/pages/Signup.jsx`** file to a new **`src/pages/Login.jsx`** file and adjust the import and component name. Also, add a new import for the **`useAuth`** hook:

```
import { login } from '../api/users.js'
import { useAuth } from '../contexts/AuthContext.jsx'
export function Login() {
```

9. Next, edit **`src/pages/Login.jsx`**, add the **`useAuth`** hook, adjust the **`signupMutation`** to call login, set the token, and navigate to the index page upon successfully logging in:

```
  const [, setToken] = useAuth()
  const loginMutation = useMutation({
    mutationFn: () => login({ username, password }),
    onSuccess: (data) => {
      setToken(data.token)
      navigate('/')
    },
    onError: () => alert('failed to login!'),
  })
  const handleSubmit = (e) => {
    e.preventDefault()
```

```
      loginMutation.mutate()
    }
```

10. Adjust the submit button, as follows:

```
      <input
        type='submit'
        value={loginMutation.isPending ? 'Logging in...' : 'Log In'}
        disabled={!username || !password || loginMutation.isPending}
      />
```

11. Edit **`src/App.jsx`** and import the **`Login`** page:

```
import { Login } from './pages/Login.jsx'
```

12. Lastly, define the **`/login`** route, as follows:

```
  {
    path: '/login',
    element: <Login />,
  },
```

With that, our signup and login pages are working properly, but we still need to link to the login page and show the currently logged-in user on the index page. Let's do that now.

Using the stored JWT and implementing a simple logout

In this section, we are going to check if the user is logged in already by checking if there is a valid JWT stored in the context. Then, we are going to use the auth context hook to log our user out again by simply removing the token from it. This is not a full logout as the JWT is still technically valid. For a full logout, we would have to invalidate the token in the back-end (for example, by blacklisting that token in the authentication service database). This process is called **token revocation**.

Let's start using the stored JWT and implement a simple logout:

1. Install the **`jwt-decode`** library in the root of our project (the frontend):

```
$ npm install jwt-decode@4.0.0
```

2. Edit **`src/components/Header.jsx`** and import the **`jwtDecode`** function and the **`useAuth`** hook:

```
import { jwtDecode } from 'jwt-decode'
import { useAuth } from '../contexts/AuthContext.jsx'
```

3. Get the token from the **`useAuth`** hook in the **`Header`** component:

```
export function Header() {
  const [token, setToken] = useAuth()
```

4. Add a check for if the token is properly set. If it is, parse the token and render the user ID from it:

```
  if (token) {
    const { sub } = jwtDecode(token)
    return (
      <div>
        Logged in as <b>{sub}</b>
```

NOTE

In this case, we are only decoding the token in one place. If this functionality is used in multiple places, it would make sense to abstract the decoding into a separate hook.

5. Additionally, we'll show a button to log out here, which just resets the token:

```
        <br />
        <button onClick={() => setToken(null)}>Logout</button>
      </div>
    )
  }
```

6. While we're at it, let's also add a link to the login page to the header, if the user isn't logged in yet:

```
  return (
    <div>
      <Link to='/login'>Log In</Link> | <Link to='/signup'>Sign Up</Link>
    </div>
  )
```

Congratulations! We have successfully implemented a simple JWT user authentication flow. However, you may have noticed that all the users in our blog appear as their user ID, not with their username. Let's change that.

Fetching the usernames

To show the usernames instead of the user IDs, we are going to create a **`User`** component that will fetch user information from an endpoint in our backend, which we are going to create now. For now, we will only show the username, but in the future, this feature could be used to fetch other information, such as the avatar or full name of the user.

Implementing the backend endpoint

Let's get started by implementing the backend endpoint for fetching user information:

1. Edit **`backend/src/services/users.js`** and add a new function to get user information by **`id`**. As a fallback, we return the user ID if we can't find a matching user:

```
export async function getUserInfoById(userId) {
  try {
    const user = await User.findById(userId)
    if (!user) return { username: userId }
    return { username: user.username }
  } catch (err) {
    return { username: userId }
  }
}
```

   We specifically make sure we only return the username here, to avoid leaking the password or other sensitive user information!
2. Edit **`backend/src/routes/users.js`** and import the newly defined function there:

```
import { createUser, loginUser, getUserInfoById } from '../services/users.js'
```

3. Then, define a new route inside the **`userRoutes`** function, which will get a user with a specific ID. For this route, we use the plural **`users`** as we are dealing with multiple users here:

```
  app.get('/api/v1/users/:id', async (req, res) => {
    const userInfo = await getUserInfoById(req.params.id)
    return res.status(200).send(userInfo)
  })
```

4. Since we are already working on the backend, let's also change the existing **`author`** filter so that it works with usernames. Edit **`backend/src/services/posts.js`** and import the **`User`** model:

```
import { User } from '../db/models/user.js'
```

5. Refactor the **`listPostsByAuthor`** function by finding a user with the given username, then listing all posts by the user ID (if one was found):

```
export async function listPostsByAuthor(authorUsername, options) {
  const user = await User.findOne({ username: authorUsername })
  if (!user) return []
  return await listPosts({ author: user._id }, options)
}
```

Now that we have an endpoint that returns user information for a given user ID, let's use it in the frontend!

Implementing a User component to fetch and render the username

In the frontend, we are going to create a component that will fetch and render the username. React Query helps us a lot here because we don't need to worry about fetching the same user IDs multiple times – it will cache the result for us and instantly return it, instead of making another request.

Follow these steps to implement a **`User`** component:

1. First, we need to define the API function. Edit **`src/api/users.js`** and add a function to get the user info by **`id`**:

```
export const getUserInfo = async (id) => {
  const res = await fetch(`${import.meta.env.VITE_BACKEND_URL}/users/${id}`, {
    method: 'GET',
    headers: { 'Content-Type': 'application/json' },
```

```
  })
  return await res.json()
}
```

2. Create a new **`src/components/User.jsx`** file and import **`useQuery`**, **`PropTypes`**, and the API function:

```
import { useQuery } from '@tanstack/react-query'
import PropTypes from 'prop-types'
import { getUserInfo } from '../api/users.js'
```

3. Now, define the component and get the user info via the query hook:

```
export function User({ id }) {
  const userInfoQuery = useQuery({
    queryKey: ['users', id],
    queryFn: () => getUserInfo(id),
  })
  const userInfo = userInfoQuery.data ?? {}
```

4. We render the username if available and fall back to the ID otherwise:

```
  return <strong>{userInfo?.username ?? id}</strong>
}
```

5. Lastly, we define the prop types for the component:

```
User.propTypes = {
  id: PropTypes.string.isRequired,
}
```

6. Now, we can make use of the newly created component and import it in **`src/components/Header.jsx`**:

```
import { User } from './User.jsx'
```

7. Then, we can edit the existing code to render the **`User`** component instead of directly rendering the user ID:

```
        Logged in as <User id={sub} />
```

8. Next, we repeat the same process for **`src/components/Post.jsx`** and import the **`User`** component:

```
import { User } from './User.jsx'
```

9. Then, we adjust the code to render the **`User`** component:

```
Written by <User id={author} />
```

Now, our usernames will all render properly again, as shown in the following screenshot:

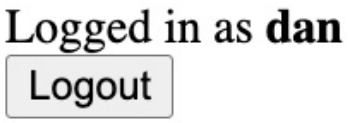

Figure 6.6 – Properly fetching and showing the username

Now that usernames show up properly, we need to do one more thing: send the JWT header when creating posts.

Sending the JWT header when creating posts

When creating a post, we don't need to send the author anymore. Instead, we need to send the JWT with the **`Authentication`** header.

Let's refactor the code so that we can do this:

1. Edit **`src/api/posts.jsx`** and adjust the **`createPost`** function so that it accepts a JWT as the first argument, which is then passed on inside an **`Authentication`** header:

```
export const createPost = async (token, post) => {
  const res = await fetch(`${import.meta.env.VITE_BACKEND_URL}/posts`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
     Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify(post),
  })
  return await res.json()
}
```

2. Edit **`src/components/CreatePost.jsx`** and import the **`useAuth`** hook:

```
import { useAuth } from '../contexts/AuthContext.jsx'
```

3. Get the JWT from the **`useAuth`** hook inside the component:

```
export function CreatePost() {
  const [token] = useAuth()
```

4. *Remove* the **`author`** state:

```
  const [author, setAuthor] = useState('')
```

5. Also, *remove* the **`author`** state from the **`createPost`** function and instead pass in the **`token`** state as the first argument:

```
    mutationFn: () => createPost(token, { title, author, contents }),
```

6. Before rendering the component, check if the user is logged in by checking if a token exists. If the user is not logged in, we tell them to log in first:

```
  if (!token) return <div>Please log in to create new posts.</div>
  return (
    <form onSubmit={handleSubmit}>
```

7. *Remove* the following code to remove the **`author`** field:

```
      <br />
      <div>
        <label htmlFor='create-author'>Author: </label>
        <input
          type='text'
          name='create-author'
          id='create-author'
          value={author}
          onChange={(e) => setAuthor(e.target.value)}
        />
      </div>
```

Now, creating a post works successfully again! It stores the user ID of the currently logged-in user in the database as the author and resolves it to the username when showing the post.

Next, we'll learn about advanced token handling.

Advanced token handling

You may have noticed that our simple authentication solution is still missing some features that a fully-fledged solution should have, such as the following:

- Using asymmetric keys for the tokens so that we can verify the authenticity (using the public key) without exposing our secret (the private key) to all services. Up until now, we have been using a symmetric key, which means that we need the same secret to generate and verify a JWT.
- Storing tokens in safe `httpOnly` cookies so that they can be accessed again, even when the page is refreshed or closed.
- Invalidating tokens after logging out on the backend.

Implementing these things requires a lot of effort manually, so it is best practice to use an authentication solution such as Auth0 or Firebase Auth. These solutions work similarly to our simple JWT implementation, but they provide an external authentication service to create and handle the tokens for us. This chapter intended to introduce how those providers work behind the scenes so that you can easily understand and integrate any of the providers as you see fit in your projects.

So far, all users have been considered equal, with everyone being allowed to create posts, but only update and delete their own posts. For a public blog, it would be good to have a way for administrators to delete other people's posts to moderate the content on the platform. A good way to add roles is to store and fetch them from the database. While adding roles in the JWT is technically possible, it has some downsides, such as the need to invalidate existing tokens and create a new token when the roles change.

Summary

In this chapter, we learned how JWTs work in depth. First, we learned about the theory of authentication and JWTs, and how to manually create them. Then, we implemented login, signup, and authenticated routes in the backend. Next, we integrated these routes in the frontend by creating new pages and routing between them using React Router. Finally, we wrapped up this chapter by learning about advanced token handling and

giving pointers on more things to learn about authentication and role management.

In the next chapter, ***Chapter 7***, *Improving the Load Time Using Server-Side Rendering*, we are going to learn how to implement server-side rendering to improve the initial load time of our blog. We are already doing a lot of requests on the first load (fetching all blog posts, then the usernames of each author). We can bundle them together by doing this on the backend.